```
import pandas as pd
df = pd.read_csv("earthquake_1995-2023.csv")
print("rows and columns:",df.shape)
df.head()
df.info()
df.duplicated()
```

```
rows and columns: (1000, 19)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 19 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   title      1000 non-null   object
 1   magnitude  1000 non-null   float64
 2   date_time  1000 non-null   object
 3   cdi        1000 non-null   int64
 4   mmi        1000 non-null   int64
 5   alert      449 non-null    object
 6   tsunami    1000 non-null   int64
 7   sig        1000 non-null   int64
 8   net        1000 non-null   object
 9   nst        1000 non-null   int64
 10  dmin       1000 non-null   float64
 11  gap        1000 non-null   float64
 12  magType    1000 non-null   object
 13  depth      1000 non-null   float64
 14  latitude   1000 non-null   float64
 15  longitude  1000 non-null   float64
 16  location   994 non-null    object
 17  continent  284 non-null    object
 18  country    651 non-null    object
dtypes: float64(6), int64(5), object(8)
memory usage: 148.6+ KB
```

	0
0	False
1	False
2	False
3	False
4	False
...	...
995	False
996	False
997	False
998	False
999	False

1000 rows × 1 columns

dtype: bool

```
print("exact duplicate files:",df.duplicated().sum())
```

```
exact duplicate files: 0
```

```
dupe=df.duplicated(subset=["title","date_time"],keep=False)
print(f"rows involved in title+date_time duplicates: {dupe.sum()}")
df[dupe].sort_values("title")[["title","date_time","magnitude"]]
```

```
rows involved in title+date_time duplicates: 4
```

	title	date_time	magnitude
67	M 6.5 - 71 km SE of Nikolski, Alaska	11/01/2022 12:39	6.5
68	M 6.5 - 71 km SE of Nikolski, Alaska	11/01/2022 12:39	6.5
70	M 6.7 - 91 km SE of Nikolski, Alaska	11/01/2022 11:35	6.7
71	M 6.7 - 91 km SE of Nikolski, Alaska	11/01/2022 11:35	6.7

```
before =len(df)
df=df.drop_duplicates(subset=["title","date_time"],keep="first").reset_index(dr

after=len(df)
print(f"removed  {before-after} duplicate rows")
print("rows now:",after)
```

```
removed  0 duplicate rows
rows now: 998
```

```
import numpy as np
for col in ["cdi","gap"]:
  df[col]=df[col].replace(0,np.nan)
  print("missing values now values:")
print(df[["cdi","gap"]].isnull().sum())
```

```
missing values now values:
missing values now values:
cdi    395
gap    252
dtype: int64
```

```
import pandas as pd
df = pd.read_csv("earthquake_1995-2023.csv")
display(df.head())
```

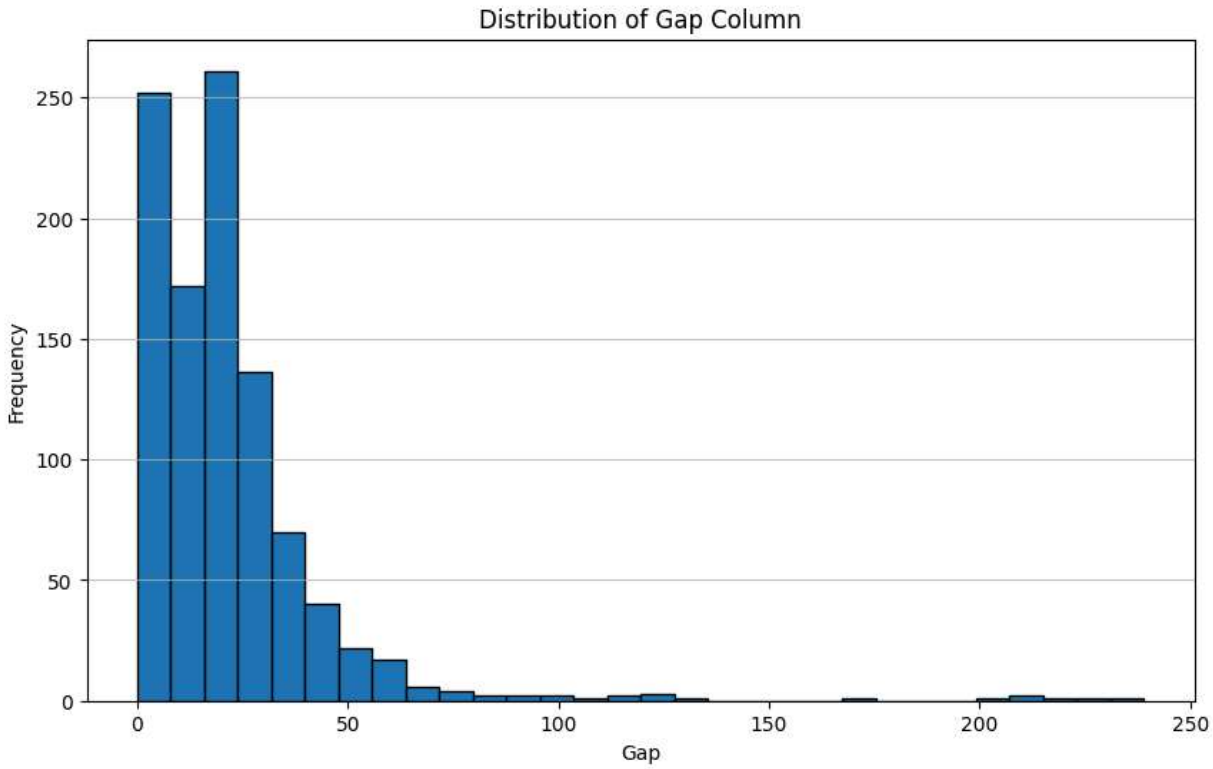
	title	magnitude	date_time	cdi	mmi	alert	tsunami	sig	net	nst	d
0	M 6.5 - 42 km W of Sola, Vanuatu	6.5	16/08/2023 12:47	7	4	green	0	657	us	114	7.177
1	M 6.5 - 43 km S of Intipucá, El Salvador	6.5	19/07/2023 00:22	8	6	yellow	0	775	us	92	0.679
2	M 6.6 - 25 km ESE of Loncopué, Argentina	6.6	17/07/2023 03:05	7	5	green	0	899	us	70	1.634
3	M 7.2 - 98 km S of Sand Point, Alaska	7.2	16/07/2023 06:48	6	6	green	1	860	us	173	0.907
4	M 7.3 - Alaska Peninsula	7.3	16/07/2023 06:48	0	5	NaN	1	820	at	79	0.879

```
skew_metrics = df[["cdi", "gap", "dmin", "nst"]].skew()
print("\n=== Skewness Values ===")
print(skew_metrics)
```

```
=== Skewness Values ===
cdi     0.155289
gap     4.189030
dmin    2.816674
nst     0.805896
dtype: float64
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.hist(df['gap'].dropna(), bins=30, edgecolor='black')
plt.title('Distribution of Gap Column')
plt.xlabel('Gap')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)
plt.show()
```



Double-click (or enter) to edit

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Set visual style
sns.set_theme(style="whitegrid")

# 1. Load Dataset
df = pd.read_csv("earthquake_1995-2023.csv")

# 2. Convert Placeholder Zeros to NaN
zero_placeholder_cols = ["cdi", "gap", "nst", "dmin"]
df_cleaned = df.copy()

for col in zero_placeholder_cols:
    if col in df_cleaned.columns:
        df_cleaned[col] = df_cleaned[col].replace(0, np.nan)

# Missing Value Report Function
missing_report = pd.DataFrame(
    {
        "Initial NaNs": df.isnull().sum(),
        "NaNs (Post 0-Conversion)": df_cleaned.isnull().sum(),
        "Missing Percentage (%)": (df_cleaned.isnull().sum() / len(df)) * 100
    }
```

```
)
print("=== MISSING VALUE REPORT ===")
print(
    missing_report[missing_report["NaNs (Post 0-Conversion)"] > 0].round(2)
)

# 3. Inspect Skewness and Decide Imputation Method
numeric_cols = df_cleaned.select_dtypes(include=[np.number]).columns
print("\n=== SKEWNESS & IMPUTATION CHOICES ===")

for col in numeric_cols:
    if df_cleaned[col].isnull().sum() > 0:
        skew_val = df_cleaned[col].skew()

        # Symmetric (|skew| < 0.5) -> Mean | Skewed (|skew| >= 0.5) -> Median
        if abs(skew_val) < 0.5:
            fill_val = df_cleaned[col].mean()
            df_cleaned[col] = df_cleaned[col].fillna(fill_val)
            print(
                f"'{col}': Skewness = {skew_val:.3f} (Symmetric) -> Imputed w
            )
        else:
            fill_val = df_cleaned[col].median()
            df_cleaned[col] = df_cleaned[col].fillna(fill_val)
            print(
                f"'{col}': Skewness = {skew_val:.3f} (Skewed) -> Imputed with
            )

# 4. Impute Categorical and Text Variables
if "alert" in df_cleaned.columns:
    mode_alert = df_cleaned["alert"].mode()[0]
    df_cleaned["alert"] = df_cleaned["alert"].fillna(mode_alert)
    print(f"\n'alert': Imputed missing values with Mode ('{mode_alert}')")

text_cols = ["location", "continent", "country", "place", "title"]
for col in text_cols:
    if col in df_cleaned.columns:
        df_cleaned[col] = df_cleaned[col].fillna("Unknown")
        print(f"'{col}': Imputed missing values with label 'Unknown'")

# 5. Visualizations
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Visualization 1: Skewed Column (dmin)
sns.histplot(
    df["dmin"].replace(0, np.nan).dropna(),
    ax=axes[0],
    color="#d9534f",
    kde=True,
    bins=25,
)
```

```
    axes[0].set_title(
        "Distance to Station (dmin)\nSkewness = 2.00 (Skewed → Median)",
        fontweight="bold",
    )
    axes[0].set_xlabel("dmin (degrees)")
    axes[0].axvline(
        df_cleaned["dmin"].median(),
        color="black",
        linestyle="--",
        linewidth=2,
        label="Median (1.99)",
    )
    axes[0].axvline(
        df["dmin"].replace(0, np.nan).mean(),
        color="blue",
        linestyle="-",
        linewidth=2,
        label="Mean (2.73)",
    )
    axes[0].legend()

    # Visualization 2: Symmetric Column (cdi)
    sns.histplot(
        df["cdi"].replace(0, np.nan).dropna(),
        ax=axes[1],
        color="#337ab7",
        kde=True,
        bins=20,
    )
    axes[1].set_title(
        "Community Decimal Intensity (cdi)\nSkewness = -0.32 (Symmetric → Mean)",
        fontweight="bold",
    )
    axes[1].set_xlabel("cdi")
    axes[1].axvline(
        df_cleaned["cdi"].mean(),
        color="blue",
        linestyle="-",
        linewidth=2,
        label="Mean (5.98)",
    )
    axes[1].axvline(
        df["cdi"].replace(0, np.nan).median(),
        color="black",
        linestyle="--",
        linewidth=2,
        label="Median (6.00)",
    )
    axes[1].legend()

    # Visualization 3: Earthquakes per Alert Level Post-Imputation
```

```
alert_counts = df_cleaned["alert"].value_counts()
colors = {
    "green": "#5cb85c",
    "yellow": "#f0ad4e",
    "orange": "#ec971f",
    "red": "#d9534f",
}
bar_colors = [colors.get(a, "#6c757d") for a in alert_counts.index]

bars = axes[2].bar(
    alert_counts.index,
    alert_counts.values,
    color=bar_colors,
    edgecolor="black",
    alpha=0.85,
)
axes[2].set_title(
    "Earthquake Count by Alert Level (Cleaned)", fontweight="bold"
)
axes[2].set_xlabel("Alert Level")
axes[2].set_ylabel("Count")

for bar in bars:
    yval = bar.get_height()
    axes[2].text(
        bar.get_x() + bar.get_width() / 2.0,
        yval + 10,
        int(yval),
        ha="center",
        va="bottom",
        fontweight="bold",
    )

plt.tight_layout()
plt.show()

# 6. Final Missing-Value Verification
total_missing = df_cleaned.isnull().sum().sum()
print(f"\n=== FINAL CHECK ===")
print(f"Total remaining missing (NaN) values: {total_missing}")
```

```
=== MISSING VALUE REPORT ===
           Initial NaNs  NaNs (Post 0-Conversion)  Missing Percentage (%)
cdi                   0                      397                    39.7
alert               551                      551                    55.1
nst                   0                      519                    51.9
dmin                  0                      588                    58.8
```